



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 2

Percepción del entorno

Estimación de paredes por regresión lineal de mínimos cuadrados

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: ROS 2 Humble · Coppeliasim · Pioneer 3-DX

Índice

1. Objetivo	3
2. Implementación	3
3. Pruebas	4
4. Comentarios finales	5

1. Objetivo

Sustituir la función `averageRangeInWindow()` de Lab-MR 1-II por una función `extractWall()` que estime cada pared del pasillo como una recta $y = mx + b$ mediante **regresión lineal por mínimos cuadrados** sobre los puntos láser. Crear el nodo `corr_nav_wall` (basado en el de la práctica anterior), adaptar el bucle de control para usar las dos rectas en el cálculo del punto objetivo de Pure Pursuit, compilar y lanzar con `ros2 launch seg_tray corr_nav_wall_launch.py`.

2. Implementación

El nodo `corr_nav_wall_node` se ha derivado del de Lab-MR 1-II reemplazando `averageRangeInWindow()` por `extractWall()` y eliminando el servicio que ya no se necesita. La estructura general (callback de láser, callback de odometría, bucle de control a 40 Hz que publica `cmd_vel`) se mantiene. Las decisiones no obvias son:

Rangos angulares estrechos. La pared izquierda se extrae del sector $[60^\circ, 120^\circ]$ y la derecha del sector $[-120^\circ, -60^\circ]$. Rangos más amplios (e.g. $\pm 30^\circ$ a $\pm 150^\circ$) incluyen lecturas de las esquinas del pasillo, que sesgan la regresión y hacen oscilar la orientación estimada.

Filtrado de lecturas inválidas. Se descartan rangos no finitos o no positivos antes de convertir a cartesianas. Si quedan menos de 2 puntos válidos, la pared se marca como no detectada ($b = +\infty$).

Protección frente a paredes verticales. La fórmula de mínimos cuadrados para m tiene como denominador $n \sum x_i^2 - (\sum x_i)^2$, que tiende a cero cuando los puntos comparten la misma x (pared perpendicular al eje del robot). En ese caso se devuelve $m = 0$ y $b = \bar{y}$, lo que equivale a tratar la pared como horizontal y evita un NaN.

Punto objetivo analítico. Conocidas (m_L, b_L) y (m_R, b_R) , el centro del pasillo a la distancia de look-ahead L es directamente

$$y_{\text{goal}} = \frac{m_L + m_R}{2} L + \frac{b_L + b_R}{2},$$

y la curvatura de Pure Pursuit es $\kappa = 2 y_{\text{goal}} / L^2$. No hace falta promediar lecturas a izquierda y derecha como en Lab-MR 1-II: el centrado y la orientación quedan absorbidos por el ajuste lineal.

Cláusula de guarda al perder una pared. Si b_L o b_R no son finitos (e.g. en una curva el sector angular deja de ver una pared), se publica $v = \omega = 0$ en vez de calcular comandos erráticos. Es una protección de seguridad más que un modo de curva.

3. Pruebas

Tras compilar e incluir el ejecutable en `CMakeLists.txt` y crear el launch file copiado del de Lab-MR 1-II, el nodo se lanza y el robot navega por el centro del pasillo en el escenario de CoppeliaSim (Figura 1).

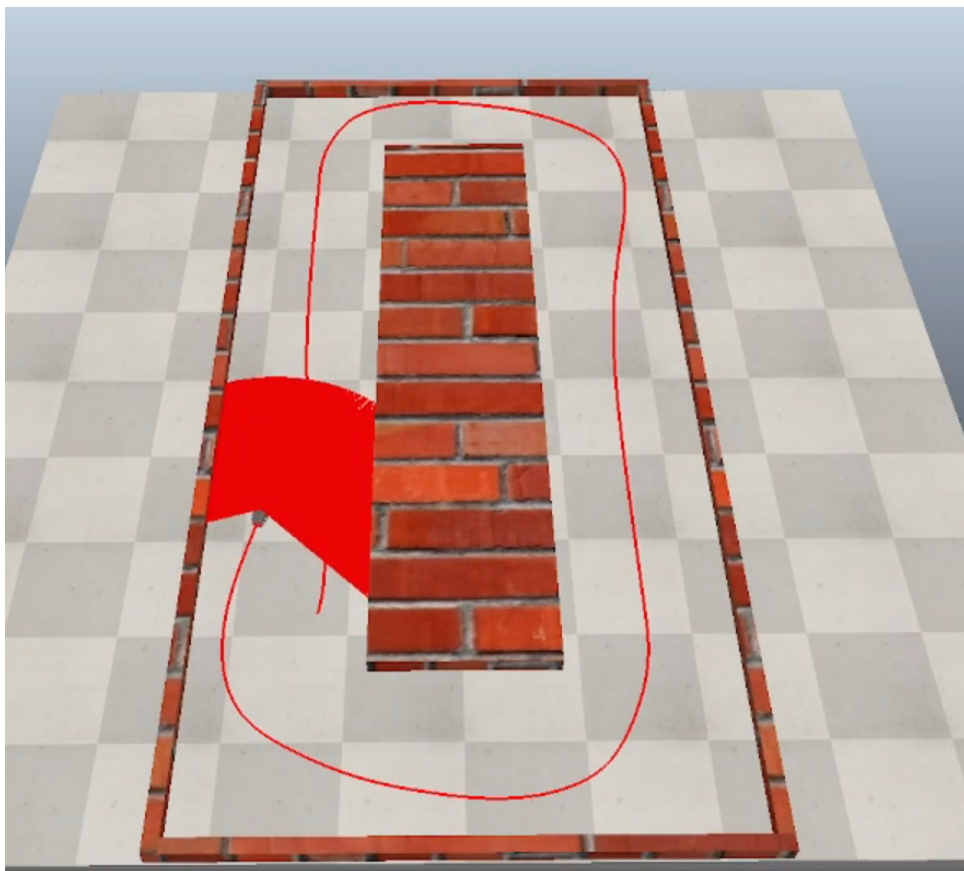


Figura 1: Resultado de la navegación con estimación de paredes en CoppeliaSim: el robot Pioneer P3DX sigue el pasillo de forma centrada usando los modelos de pared izquierda y derecha.

En tramos rectos, el comportamiento es notablemente más suave que con el método de ventanas angulares de Lab-MR 1-II porque la regresión integra todas las lecturas del sector y filtra el ruido individual. La orientación del robot se obtiene directamente de las pendientes m_L y m_R , sin necesidad de la heurística de comparar lecturas a $\pm 15^\circ$ que usábamos antes. La velocidad de cruce se ha

podido subir a $0,5 \text{ m s}^{-1}$ frente a los $0,25 \text{ m s}^{-1}$ de la práctica anterior, gracias a la mayor estabilidad del estimador.

En las curvas, esta versión no implementa un modo dedicado: cuando una pared sale del sector de detección el intercepto se vuelve infinito y la cláusula de guarda detiene el robot. La integración con el modo curva de Lab-MR 1-II queda como mejora natural pero no la pide el enunciado de esta práctica.

4. Comentarios finales

La sustitución de `averageRangeInWindow()` por `extractWall()` cumple lo que pide el enunciado: las paredes se modelan como rectas $y = mx + b$ por mínimos cuadrados, el punto objetivo se calcula analíticamente como el centro del pasillo a la distancia de look-ahead, y el control mantiene Pure Pursuit. La estimación por regresión es más robusta al ruido que el promedio en ventanas estrechas, lo que se traduce en trayectorias más suaves y velocidad de cruce más alta en tramos rectos.